

30 Câu Hỏi Phỏng Vấn React Fresher 2026

Bộ tài liệu thực chiến — Gợi ý trả lời, bẫy hay gặp, mẫu trả lời chuẩn

Tài liệu miễn phí từ **CodeGym**. Phiên bản 1.0 — Cập nhật tháng 5/2026.

Ai nên đọc tài liệu này?

- Sinh viên IT năm 3–4 chuẩn bị xin **intern / fresher React Developer**.
- Người đang chuyển ngành sang frontend, đã học React 2–4 tháng và sắp đi phỏng vấn lần đầu.
- Dev đã có 6 tháng kinh nghiệm muốn ôn lại căn bản trước khi nhảy việc.

Tài liệu **không dành cho** người muốn học React từ con số 0. Bạn cần đã build được tối thiểu 1 project React (todo app, blog, e-commerce mini) trước khi đọc.

Cách dùng tài liệu hiệu quả nhất

1. **Đừng học vẹt cả 30 câu.** Đọc từng câu, tự trả lời ra giấy/miệng trước, rồi đối chiếu mẫu. Cảm giác "biết mà nói không trôi" là dấu hiệu bạn chưa thực sự hiểu — quay lại đọc tài liệu/code thử.
2. **Mỗi câu có 4 phần:**
 - 💡 **Cấu trúc trả lời** — khung để bạn không lan man, mất điểm.
 - ✅ **Mẫu trả lời** — viết theo giọng tự nhiên, không "đọc thuộc".
 - ⚠️ **Bẫy & câu hỏi đào sâu** — interviewer thường hỏi tiếp gì.
 - (Khi cần) đoạn code minh họa.
3. **Luyện nói thành tiếng** ít nhất 3 lần/câu. Hiểu trong đầu khác hoàn toàn với nói ra được.
4. **Đánh dấu câu bạn vẫn cảm thấy yếu** — đó chính là điểm bạn cần lên Google/YouTube đào sâu thêm.

Mục lục

- **Phần A** — JavaScript core (8 câu) | Câu 1–8
 - **Phần B** — React fundamentals (10 câu) | Câu 9–18
 - **Phần C** — Hooks & quản lý state (7 câu) | Câu 19–25
 - **Phần D** — HTTP, tooling, deployment (5 câu) | Câu 26–30
 - **Phụ lục** — Tips trước buổi phỏng vấn + lộ trình tự luyện
-

PHẦN A — JavaScript Core (Câu 1–8)

Interviewer hỏi JS trước React vì nếu JS yếu, học React chỉ là copy syntax không hiểu gì.

Câu 1. Sự khác nhau giữa `var`, `let`, `const` là gì? Bạn dùng cái nào trong dự án thật?

💡 Cấu trúc trả lời: scope → khả năng gán lại → hoisting → khuyến nghị thực tế.

✅ Mẫu trả lời:

"Cả 3 đều khai báo biến nhưng khác về scope và behavior. `var` là function-scoped và bị hoisted lên đầu function với giá trị `undefined` — đây là lý do gây nhiều bug khó debug. `let` và `const` là block-scoped (trong cặp `{}`) và bị hoisted nhưng nằm trong temporal dead zone, nên truy cập trước khi khai báo sẽ throw `ReferenceError`. `const` thì không gán lại được, nhưng nếu là object/array vẫn mutate được nội dung bên trong. Trong dự án thật mình mặc định dùng `const`, chỉ chuyển sang `let` khi rõ ràng cần gán lại, và không dùng `var` nữa từ ES6."

⚠ Bẫy hay gặp tiếp theo:

- "Vậy `const arr = [1,2]; arr.push(3);` có lỗi không?" → Không, vì `const` chỉ khoá reference, không khoá nội dung.
- "Temporal dead zone là gì?" → Giai đoạn từ lúc scope bắt đầu đến lúc biến `let` / `const` được khai báo — truy cập trong khoảng này sẽ lỗi.

Câu 2. Hoisting là gì? Cho ví dụ.

💡 Cấu trúc trả lời: định nghĩa → khác biệt giữa function declaration và variable → ví dụ minh hoạ.

✅ Mẫu trả lời:

"Hoisting là cơ chế JS đưa khai báo (declaration) lên đầu scope trước khi code chạy. Function declaration được hoisted cả định nghĩa, nên gọi function trước dòng khai báo vẫn chạy. Variable thì khác: `var` được hoist với giá trị `undefined`, còn `let` / `const` được hoist nhưng nằm trong temporal dead zone."

```
console.log(a); // undefined – không lỗi vì var được hoist
var a = 5;

console.log(b); // ReferenceError – let trong TDZ
let b = 5;

sayHi(); // chạy bình thường – function declaration hoisted
function sayHi() { console.log("hi"); }
```

⚠ **Bẫy:** "Function expression có hoisted không?" → Có hoisted phần biến nhưng giá trị là `undefined`, nên gọi `myFunc()` trước khi gán sẽ lỗi `myFunc is not a function`.

Câu 3. Closure là gì? Cho ví dụ thực tế.

💡 **Cấu trúc trả lời:** định nghĩa ngắn → cơ chế (lexical scope) → 1 use case thực tế.

✅ **Mẫu trả lời:**

"Closure là khả năng một function nhớ và truy cập được biến từ scope cha của nó, kể cả khi function cha đã chạy xong. Đây là hệ quả tự nhiên của lexical scoping trong JS. Ứng dụng phổ biến là tạo private variable, hoặc trong React là cách `useState` giữ giá trị state qua các lần render."

```
function counter() {
  let count = 0; // biến "private"
  return () => ++count;
}
const tick = counter();
tick(); // 1
tick(); // 2 – count vẫn được nhớ
```

⚠ **Bẫy:** "Closure trong React hooks gây bug như nào?" → Stale closure: callback trong `useEffect` bắt giá trị state cũ. Sửa bằng `useRef` hoặc dùng functional update: `setCount(prev => prev + 1)`.

Câu 4. `==` và `===` khác nhau ở đâu? Khi nào nên dùng cái nào?

✅ **Mẫu trả lời:**

"`==` so sánh có ép kiểu (coercion), nên `0 == false` là `true`, `'5' == 5` cũng `true`. Còn `===` so sánh nghiêm ngặt cả kiểu lẫn giá trị — `0 === false` là `false`. Trong dự án mình luôn dùng `===` để tránh bug khó debug. Trường hợp duy nhất mình dùng `==` là check `val == null` để bắt cả `null` và `undefined` cùng lúc — nhưng cái này có thể viết `val === null || val === undefined` cũng được."

⚠ Bẫy: "Tại sao `[] == false` là `true`?" → JS coerce `[]` thành `""` rồi thành `0`, còn `false` cũng coerce thành `0` → bằng nhau. Bài học: tránh `==`.

Câu 5. Promise là gì? `async/await` khác gì với `.then()` ?

💡 Cấu trúc trả lời: định nghĩa Promise → 3 trạng thái → `async/await` là syntactic sugar.

✅ Mẫu trả lời:

"Promise là object đại diện cho một giá trị bất đồng bộ trong tương lai. Nó có 3 trạng thái: `pending`, `fulfilled`, `rejected`. Có thể chain bằng `.then().catch()`, hoặc dùng `async/await` để viết code bất đồng bộ trông như đồng bộ, dễ đọc hơn nhiều. `async/await` thực chất chỉ là syntactic sugar bọc lên Promise — `await` chờ Promise resolve, lỗi thì bắt bằng `try/catch`."

```
// Cách 1: .then
fetch('/api/user').then(r => r.json()).then(data => console.log(data));

// Cách 2: async/await - dễ đọc hơn
async function getUser() {
  try {
    const res = await fetch('/api/user');
    const data = await res.json();
    console.log(data);
  } catch (err) { console.error(err); }
}
```

⚠ Bẫy: "Khi nào dùng `Promise.all` vs `Promise.allSettled`?" → `Promise.all` reject ngay khi 1 promise fail; `Promise.allSettled` đợi tất cả xong, không quan tâm fail hay success.

Câu 6. Event loop hoạt động như thế nào?

💡 Cấu trúc trả lời: call stack → web APIs → callback queue + microtask queue → loop logic.

✓ Mẫu trả lời:

"JS chạy single-thread, dùng event loop để xử lý bất đồng bộ. Code đồng bộ đẩy vào call stack chạy ngay. Các tác vụ bất đồng bộ (setTimeout, fetch, DOM event) được giao cho Web APIs xử lý, xong xuôi thì callback được đẩy vào queue. Có 2 queue chính: microtask queue (Promise.then, queueMicrotask) — ưu tiên cao, và macrotask queue (setTimeout, setInterval). Event loop liên tục kiểm tra call stack rỗng chưa, rỗng thì lấy task từ microtask queue trước, rồi mới đến macrotask."

⚠ Bẫy thường gặp: cho đoạn code trộn `setTimeout` + `Promise.resolve().then` và hỏi thứ tự output. Quy tắc: microtask luôn trước macrotask.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// Output: 1, 4, 3, 2
```

Câu 7. Spread operator và rest parameter khác nhau như nào?

✓ Mẫu trả lời:

"Cả 2 đều dùng dấu `...` nhưng ngữ cảnh khác nhau. Spread *unpack* (rải) phần tử từ array/object ra. Rest *pack* (gom) nhiều phần tử thành 1 array."

```
// Spread — rải ra
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1,2,3,4,5]

const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 }; // {a:1, b:2, c:3}

// Rest — gom lại
function sum(...nums) { return nums.reduce((a,b) => a+b, 0); }
sum(1, 2, 3); // 6
```

⚠ Bẫy: "Spread object có deep clone không?" → Không, chỉ shallow clone. Object lồng trong vẫn dùng chung reference. Deep clone dùng `structuredClone()` hoặc lodash `cloneDeep`.

Câu 8. `this` trong JS hoạt động như thế nào? Arrow function khác gì?

💡 Cấu trúc trả lời: `this` phụ thuộc context gọi → 4 quy tắc cơ bản → arrow function không có `this` riêng.

✅ Mẫu trả lời:

"`this` của function thường được xác định lúc gọi, không phải lúc định nghĩa. 4 quy tắc cơ bản: gọi trực tiếp `func()` → `this` là `undefined` ở strict mode (window ở non-strict); gọi như method `obj.func()` → `this` là `obj`; gọi với `new` → `this` là object mới; gọi với `call/apply/bind` → `this` là tham số đầu. Còn arrow function không có `this` riêng — nó kế thừa `this` từ scope bao quanh tại lúc định nghĩa. Đây là lý do trong React class component cũ ta hay phải `this.handleClick = this.handleClick.bind(this)`, còn arrow function thì khỏi cần."

⚠️ Bẫy: "Trong `setTimeout(function(){ this... }, 1000)` thì `this` là gì?" → `undefined` (strict) hoặc `window`. Fix: dùng arrow function hoặc `bind`.

PHẦN B — React Fundamentals (Câu 9–18)

Câu 9. Virtual DOM là gì? Tại sao React lại nhanh hơn thao tác DOM trực tiếp?

💡 Cấu trúc trả lời: vấn đề của DOM thật → virtual DOM giải quyết bằng cách nào → reconciliation/diffing.

✅ Mẫu trả lời:

"DOM thật rất chậm khi cập nhật, vì mỗi thay đổi có thể trigger reflow + repaint cả layout. React tạo ra Virtual DOM — một bản sao nhẹ của DOM thật ở dạng JS object. Khi state thay đổi, React tạo virtual DOM mới, so sánh (diff) với bản cũ bằng thuật toán reconciliation, rồi chỉ apply những thay đổi tối thiểu lên DOM thật. Cách này hạn chế thao tác DOM trực tiếp — chỗ thực sự đắt — nên app nhanh và mượt hơn."

⚠ Bẫy:

- "Virtual DOM có luôn nhanh hơn DOM trực tiếp không?" → Không. Với app nhỏ hoặc thao tác đơn giản, DOM trực tiếp nhanh hơn. Virtual DOM hữu ích khi UI phức tạp, nhiều update song song.
- "React 19 với compiler có thay đổi gì?" → React Compiler tự optimize re-render, giảm nhu cầu `useMemo` / `useCallback` manual.

Câu 10. JSX là gì? Trình duyệt có hiểu JSX không?

✅ Mẫu trả lời:

"JSX là syntax extension của JS, cho phép viết HTML-like trong code JS. Trình duyệt không hiểu JSX trực tiếp — Babel (hoặc SWC trong các tool mới) compile JSX thành lời gọi `React.createElement(type, props, children)`. Ví dụ `<div className='box'>Hi</div>` được compile thành `React.createElement('div', {className: 'box'}, 'Hi')`."

⚠ Bẫy:

- "Tại sao là `className` mà không phải `class`?" → Vì `class` là từ khoá reserved trong JS.

- "Có dùng React được mà không cần JSX không?" → Có. Viết trực tiếp `React.createElement` — nhưng dài và khó đọc.

Câu 11. Props và state khác nhau như nào?

✓ Mẫu trả lời:

"Props là dữ liệu cha truyền xuống con — bất biến trong component nhận, chỉ component cha mới sửa được. State là dữ liệu nội bộ của component, do chính component đó quản lý qua `useState` (hoặc `class state`). State thay đổi → component re-render. Quy tắc kinh nghiệm: dữ liệu chỉ component này dùng → đặt state nội bộ; dữ liệu nhiều component cùng cần → nâng lên component cha (lifting state up) hoặc dùng context/store."

⚠ Bẫy: "Có nên modify props trực tiếp không?" → Không bao giờ. Props là read-only. Muốn thay đổi → gọi callback do cha truyền xuống.

Câu 12. Controlled vs Uncontrolled component là gì?

✓ Mẫu trả lời:

"Controlled component là form input mà giá trị được React state quản lý — `value` lấy từ state, `onChange` cập nhật state. Uncontrolled component dùng DOM tự lưu giá trị, React truy cập qua `ref`. Trong dự án thật mình mặc định dùng controlled vì dễ validate, dễ reset, dễ sync với state khác. Uncontrolled chỉ dùng khi cần tích hợp với library cũ hoặc form rất đơn giản."

```
// Controlled
const [name, setName] = useState('');


```

⚠ Bẫy: "Form lớn 20 input dùng controlled có chậm không?" → Có thể chậm nếu mỗi keystroke re-render cả form. Giải pháp: chia nhỏ component, hoặc dùng `react-hook-form` (kết hợp uncontrolled + ref).

Câu 13. Component lifecycle trong React function component?

`useEffect` thay thế những method nào?

✅ Mẫu trả lời:

"Trong function component, vòng đời được thể hiện qua `useEffect`. Ba thời điểm chính: khi component mount (xuất hiện lần đầu), khi update (props/state đổi), khi unmount (rời khỏi UI).

- `useEffect(() => {...}, [])` chạy 1 lần khi mount — thay thế `componentDidMount`.
- `useEffect(() => {...}, [dep])` chạy khi `dep` đổi — thay thế `componentDidUpdate`.
- Return function trong `useEffect` chạy khi unmount hoặc trước lần effect tiếp theo — thay thế `componentWillUnmount`."

```
useEffect(() => {
  const id = setInterval(() => console.log('tick'), 1000);
  return () => clearInterval(id); // cleanup khi unmount
}, []);
```

⚠ Bẫy: "Trong React 18+ strict mode, `useEffect` chạy 2 lần lúc mount, đúng không?" → Đúng ở dev mode. Đây là chủ ý để phát hiện code không idempotent. Production chạy 1 lần.

Câu 14. Key trong list là gì? Đặt `key={index}` có vấn đề gì?

✅ Mẫu trả lời:

"Key giúp React xác định phần tử nào trong list bị thay đổi/thêm/xoá, để reconciliation chính xác. Key nên là id ổn định, duy nhất, không đổi giữa các lần render. Đặt `key={index}` chạy được nhưng có bug khi list reorder hoặc xoá item ở giữa — React nhầm lẫn item nào là item nào, dẫn đến state nội bộ của child bị gán nhầm hoặc input giữ giá trị cũ sai chỗ."

⚠ Ví dụ bug thực tế:

```
// User type 'abc' vào item thứ 2, rồi xoá item thứ 1
// Với key={index}, React nghĩ item 2 cũ chính là item 1 mới
// → giữ lại state input 'abc' ở vị trí sai
```

⚠ Bẫy: "Nếu data không có id thì sao?" → Tạo id ổn định lúc nhận data (uuid, hoặc hash từ nội dung).
Tránh `Math.random()` mỗi render → key sẽ đổi mỗi lần.

Câu 15. Conditional rendering có những cách nào?

✅ Mẫu trả lời:

"Có 4 cách chính: ternary `cond ? <A/> : <B/>`, short-circuit `cond && <A/>`, gán JSX vào biến rồi return, và sớm return `if (cond) return null`. Mỗi cách phù hợp ngữ cảnh khác nhau. Lưu ý bẫy với `&&`: nếu `cond` là số `0`, JSX sẽ render ra số `0` thay vì ẩn — nên ép boolean trước: `!!arr.length && <List/>`."

```
{count > 0 && <Badge count={count}/>} // 0 sẽ render '0' - bẫy
{count > 0 ? <Badge count={count}/> : null} // an toàn hơn
```

Câu 16. `React.Fragment` và `<>` để làm gì?

✅ Mẫu trả lời:

"Component React phải return một root element duy nhất. Fragment cho phép gom nhiều element mà không tạo thêm DOM node thừa. Dùng `<></>` là shorthand, nhưng nếu cần truyền `key` (vd trong list) thì phải dùng `<React.Fragment key={id}>...</React.Fragment>`."

⚠ Bẫy: "Tại sao không dùng `<div>` cho gọn?" → Tạo wrapper div không cần thiết làm phá CSS layout (flex/grid của cha), gây bug khó tìm.

Câu 17. Phân biệt `React.memo`, `useMemo`, `useCallback`.

✅ Mẫu trả lời:

"Cả 3 đều để optimize, dùng đúng chỗ. `React.memo` bọc component — chỉ re-render khi props đổi (so sánh shallow). `useMemo` cache giá trị tính toán nặng giữa các lần render. `useCallback` cache function reference — thường dùng khi truyền function xuống component memoized để không phá memoization."

```
const expensive = useMemo(() => heavyCalc(data), [data]);
const handleClick = useCallback(() => doSth(id), [id]);
const MemoChild = React.memo(Child);
<MemoChild onClick={handleClick} />
```

⚠ **Bẫy:** "Có nên bọc memo/useMemo/useCallback cho mọi thứ không?" → Không. Bản thân chúng cũng tốn chi phí. Chỉ dùng khi đã đo bằng React DevTools Profiler và xác định đúng bottleneck. React 19 với compiler tự lo phần này.

Câu 18. Lifting state up là gì? Cho ví dụ.

✅ **Mẫu trả lời:**

"Khi 2+ component anh em cần chia sẻ cùng 1 state, ta nâng state đó lên component cha chung gần nhất, rồi truyền xuống qua props. Đây là pattern cơ bản nhất trong React trước khi nghĩ đến context hay Redux/Zustand."

```
function Parent() {
  const [search, setSearch] = useState('');
  return (
    <>
      <SearchInput value={search} onChange={setSearch} />
      <ResultList query={search} />
    </>
  );
}
```

⚠ **Bẫy:** "Khi nào không nên lifting nữa?" → Khi state phải đi qua 4–5 tầng props (prop drilling). Lúc đó cần nhắc Context API hoặc state library.

PHẦN C — Hooks & Quản Lý State (Câu 19–25)

Câu 19. `useState` hoạt động như thế nào? Tại sao set state lại không có hiệu lực ngay lập tức?

✅ Mẫu trả lời:

"`useState` trả về một cặp `[value, setter]`. Khi gọi setter, React không cập nhật `value` ngay trong scope hiện tại — nó schedule một re-render và `value` mới chỉ có ở lần render kế tiếp. Trong cùng 1 event handler, biến `value` vẫn là giá trị cũ. Nếu cần dựa trên giá trị trước đó, dùng functional update: `setCount(prev => prev + 1)`."

```
const [count, setCount] = useState(0);
function add() {
  setCount(count + 1);
  setCount(count + 1); // vẫn dùng count cũ → chỉ +1
  // Sửa: setCount(p => p + 1) gọi 2 lần → +2
}
```

⚠️ Bẫy: "React có batch nhiều setState không?" → Có. React 18 mở rộng batching cho cả async callback (timeout, fetch), gọi là automatic batching.

Câu 20. `useEffect` dependency array — khi nào để rỗng, khi nào bỏ trống, khi nào có dependency?

✅ Mẫu trả lời:

"Có 3 trường hợp:

1. **Không truyền array**: effect chạy sau mỗi lần render. Hiếm khi dùng đúng.
2. **[] rỗng**: effect chỉ chạy 1 lần khi mount. Phù hợp khởi tạo, fetch lần đầu.
3. **[dep1, dep2]**: effect chạy khi mount + mỗi khi **dep** đổi. Đa số trường hợp dùng cái này.

Quy tắc vàng: mọi giá trị từ component (state, props, function) mà bạn dùng trong effect đều phải có trong dependency array. ESLint plugin **react-hooks/exhaustive-deps** tự cảnh báo. Bỏ qua dependency để 'tạm thời cho chạy' là một trong những lỗi tốn nhiều thời gian debug nhất."

⚠ **Bẫy**: "Vậy nếu deps là function/object thì sao? Nó đổi mỗi render?" → Đúng. Cần ổn định reference bằng **useCallback** / **useMemo** ở component cha, hoặc move function vào trong effect.

Câu 21. **useRef** dùng để làm gì?

✅ Mẫu trả lời:

"Có 2 use case chính: truy cập DOM trực tiếp (focus input, đo kích thước), và giữ giá trị mutable qua các lần render mà không trigger re-render. Khác với state, thay đổi **ref.current** không khiến component render lại."

```
const inputRef = useRef(null);
useEffect(() => inputRef.current?.focus(), []);

// Giữ giá trị qua render (vd: previous value)
const prevCount = useRef(count);
useEffect(() => { prevCount.current = count; }, [count]);
```

⚠ **Bẫy**: "Có nên dùng **useRef** để lưu data thay vì state không?" → Không. Nếu UI cần re-render khi data đổi, phải dùng state. Ref dành cho dữ liệu không ảnh hưởng UI.

Câu 22. Custom hook là gì? Khi nào nên tạo?

✅ Mẫu trả lời:

"Custom hook là function bắt đầu bằng `use` mà bên trong dùng các hook khác (`useState` , `useEffect` ...). Mục đích là tách logic tái sử dụng giữa nhiều component. Mình tạo custom hook khi thấy logic stateful bị lặp ở 2+ component — ví dụ `useDebounce` , `useFetch` , `useLocalStorage` ."

```
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}

// Dùng:
const debouncedSearch = useDebounce(searchInput, 300);
```

⚠ Bẫy: "Custom hook chia sẻ state giữa các component dùng nó không?" → Không. Mỗi lần gọi tạo state riêng biệt. Muốn chia sẻ phải dùng context hoặc store.

Câu 23. Context API là gì? Khi nào dùng nó thay cho Redux/Zustand?

✅ Mẫu trả lời:

"Context API giúp truyền dữ liệu xuyên qua nhiều tầng component mà không cần prop drilling. Dùng cho dữ liệu ít thay đổi và cần ở nhiều nơi: theme, auth user, locale. Không phù hợp cho state đổi liên tục vì mỗi lần Provider value đổi, mọi consumer đều re-render. State phức tạp, đổi nhiều, cần selector tinh vi → nên dùng Zustand/Redux Toolkit."

⚠ Bẫy: "Làm sao tránh re-render cả cây khi dùng context?" → Tách context theo concern (1 context cho theme, 1 cho user...), hoặc dùng `useMemo` cho value, hoặc chuyển sang Zustand với selector.

Câu 24. Sự khác biệt giữa Redux Toolkit, Zustand, Jotai? Bạn chọn cái nào và vì sao?

✅ Mẫu trả lời:

"Cả 3 đều là state library nhưng triết lý khác:

- **Redux Toolkit (RTK):** kiến trúc Flux truyền thống, store tập trung, slice/reducer/action. Mạnh cho app lớn, có DevTools mạnh, middleware (saga, thunk).
- **Zustand:** API tối giản, một hook, selector built-in, không cần Provider. Phù hợp đa số app vừa và nhỏ.
- **Jotai:** atom-based, lấy cảm hứng từ Recoil — chia state thành các atom độc lập, ai dùng atom nào thì re-render khi atom đó đổi.

Mình thường mặc định Zustand cho dự án mới vì code ít, learning curve thấp. Chỉ chuyển sang RTK nếu team đã quen Redux hoặc cần middleware phức tạp."

⚠ **Bẫy:** "Khi nào KHÔNG cần state library?" → Khi state đa phần là server data → dùng React Query / SWR là đủ. State library chỉ nên giữ client state (UI state, form state phức tạp).

Câu 25. `useReducer` khác `useState` ở đâu? Khi nào nên dùng?

✅ Mẫu trả lời:

"`useReducer` là alternative của `useState`, lấy cảm hứng từ Redux. Dùng khi state phức tạp (object nhiều field), hoặc khi logic update state có nhiều case rõ ràng. Code dễ test, dễ đọc hơn so với chuỗi `setX` rời rạc."

```
const [state, dispatch] = useReducer(reducer, initial);

function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'reset':      return { count: 0 };
    default:          throw new Error();
  }
}
```

⚠ **Bẫy:** "Khi nào nên dùng `useReducer` thay vì `useState`?" → Khi state là object có 4+ field liên quan, hoặc khi cập nhật state dựa trên state cũ + action phức tạp.

PHẦN D — HTTP, Tooling, Deployment (Câu 26–30)

Câu 26. Bạn fetch data trong React như thế nào? `useEffect` + `fetch` có vấn đề gì?

✅ Mẫu trả lời:

"Cách cơ bản là `useEffect` + `fetch` / `axios` + `useState` để lưu data + loading + error. Nhưng cách này có nhiều vấn đề khi app lớn: race condition (component unmount trước khi fetch xong), không có cache, không tự retry, mỗi component fetch riêng dù cùng endpoint. Dự án thật mình dùng **TanStack Query (React Query)** hoặc **SWR** — chúng giải quyết toàn bộ: caching, deduplication, refetch on focus, optimistic update."

```
// React Query – gọn và mạnh
const { data, isLoading, error } = useQuery({
  queryKey: ['user', id],
  queryFn: () => fetch(`/api/users/${id}`).then(r => r.json()),
});
```

⚠️ Bẫy: "Race condition trong `useEffect` + `fetch` xử lý sao nếu vẫn dùng cách cũ?" → Dùng cleanup function với cờ `cancelled`, hoặc `AbortController` để huỷ request khi unmount.

Câu 27. CORS là gì? Khi nào gặp lỗi CORS và sửa thế nào?

✅ Mẫu trả lời:

"CORS (Cross-Origin Resource Sharing) là cơ chế bảo mật của trình duyệt: khi frontend ở domain A gọi API ở domain B, trình duyệt block trừ khi server B trả về header `Access-Control-Allow-Origin` phù hợp. Lỗi CORS không phải lỗi ở frontend — phải sửa ở backend bằng cách config header cho phép origin của frontend. Lúc dev có thể dùng proxy của Vite/CRA để bypass."

⚠ **Bẫy:** "Preflight request là gì?" → Với request 'non-simple' (PUT, DELETE, có custom header...), trình duyệt gửi 1 request `OPTIONS` trước để hỏi server có cho phép không.

Câu 28. Webpack, Vite, Turbopack khác nhau như thế nào? Vì sao Vite nhanh hơn Webpack?

✅ **Mẫu trả lời:**

"Webpack là bundler truyền thống — gom toàn bộ source thành bundle trước khi dev server chạy, nên cold start chậm với project lớn. Vite tận dụng ESM native của trình duyệt: ở dev mode chỉ serve từng file khi request, không bundle. HMR cũng nhanh hơn vì chỉ swap module bị thay đổi. Khi build production, Vite vẫn bundle bằng Rollup. Turbopack là bundler thế hệ mới của Vercel, viết bằng Rust, nhanh hơn Webpack đáng kể, hiện default cho Next.js 15+."

⚠ **Bẫy:** "Tại sao production vẫn cần bundle?" → Trình duyệt request hàng nghìn module rồi sẽ chậm. Bundle giảm số request và cho phép tree-shaking, minify.

Câu 29. Lazy loading và code splitting trong React làm như thế nào?

✅ **Mẫu trả lời:**

"Dùng `React.Lazy` + `Suspense` để load component động, không gói vào bundle chính. Phù hợp khi có route hoặc feature ít người vào (admin page, modal hiếm dùng). Kết hợp với React Router thì mỗi route là 1 chunk riêng."

```
const AdminPage = React.lazy(() => import('./AdminPage'));

<Suspense fallback={<Loading />}>
  <AdminPage />
</Suspense>
```

⚠ **Bẫy:** "Lazy load có làm UX tệ không?" → Có thể có spinner mỗi lần chuyển route. Cải thiện bằng cách prefetch khi user hover link, hoặc dùng `React.Suspense` với fallback skeleton đẹp.

Câu 30. Bạn deploy app React như thế nào? CI/CD bạn từng dùng?

✅ Mẫu trả lời:

"Project mình từng làm: build bằng `npm run build` ra static files, deploy lên Vercel/Netlify (kéo trực tiếp từ GitHub, auto deploy mỗi push). Với app có backend, mình tách: frontend lên Vercel, backend lên Railway/Render hoặc VPS. Về CI/CD, mình dùng GitHub Actions: workflow chạy `npm install` → `npm test` → `npm run build` → deploy. Biến môi trường nhạy cảm (API key) đặt trong Secrets của GitHub, không commit vào repo."

⚠️ Bẫy: "Khi nào dùng Vercel khi nào dùng VPS?" → Vercel cho frontend tĩnh + serverless function; VPS khi cần backend dài hạn, websocket, hoặc kiểm soát hạ tầng.

Phụ lục — Tips Trước Buổi Phỏng Vấn

A. 5 việc cần làm trong 48h trước phỏng vấn

1. **Đọc lại JD** — gạch chân stack/từ khoá họ nhấn. Phỏng vấn sẽ đào sâu đúng những từ này.
2. **Mở GitHub project bạn ghi trong CV** — đọc lại code, chắc chắn bạn giải thích được tại sao dùng thư viện đó, kiến trúc thế nào.
3. **Tự build dự án nhỏ 1 ngày** — todo app + auth + dark mode. Để khi hỏi "bạn dùng React xong làm được gì" có ngay sample mới.
4. **Tìm hiểu công ty 30 phút** — đọc Career page, blog kỹ thuật, repo public. Một câu hỏi cuối "Em thấy team mình đang dùng X, anh chị có thể chia sẻ thêm về..." ghi điểm rất lớn.
5. **Chuẩn bị 2–3 câu hỏi ngược lại interviewer** — về team, về quy trình review code, về roadmap product. Không hỏi → trông thiếu quan tâm.

B. Cách trả lời câu "Em chưa biết / chưa làm bao giờ"

KHÔNG nói "Em không biết" rồi im lặng. Cấu trúc an toàn:

"Em chưa làm trực tiếp [X], nhưng em đoán nó tương tự [Y] mà em đã làm — vì cả 2 đều giải quyết vấn đề [Z]. Nếu được giao em sẽ đọc docs [X] và làm thử trên sample project trước khi áp vào production. Anh/chị có thể chia sẻ thêm về use case cụ thể không ạ?"

→ Vừa thật thà, vừa thể hiện tư duy giải quyết vấn đề, vừa biến câu hỏi thành cuộc trò chuyện.

C. Câu hỏi behavior thường gặp

- "Kể về một bug khó nhất em từng debug" → Chuẩn bị 1 câu chuyện cụ thể, structure STAR (Situation, Task, Action, Result).
- "Tại sao em chọn React mà không phải Vue?" → Trả lời từ trải nghiệm thực, không hạ thấp lựa chọn khác.
- "Em làm việc nhóm thế nào?" → 1 ví dụ git workflow, cách bạn review PR của bạn cùng team.

D. Lộ trình tự luyện 14 ngày trước phỏng vấn

Ngày	Nội dung
1–3	Phần A (JS core) — đọc, tự trả lời ra giấy, đối chiếu
4–7	Phần B (React fundamentals) — code lại từng ví dụ
8–10	Phần C (Hooks) — build 1 custom hook tử tế
11–12	Phần D (HTTP/Tooling/Deploy) — deploy thật 1 project lên Vercel
13	Mock interview với bạn bè (hoặc tự ghi âm trả lời)
14	Nghỉ. Đọc lại lướt câu yếu nhất. Ngủ sớm.

Tài Liệu Tiếp Theo (Sắp Ra Mắt)

Đang biên soạn:

- 30 câu phỏng vấn Java/Spring Boot Fresher
- 30 câu phỏng vấn Python/Django/Flask Fresher
- 30 câu phỏng vấn Node.js Backend Fresher
- 30 câu phỏng vấn Flutter Mobile Fresher

Để lại email ở [link form] hoặc inbox fanpage để nhận bản mới ngay khi phát hành.

Bản quyền & sử dụng Tài liệu này được biên soạn bởi **CodeGym** và phát hành miễn phí cho cộng đồng sinh viên IT Việt Nam. Bạn được phép chia sẻ nguyên bản cho bạn bè. Không được chỉnh sửa nội dung và phát hành lại dưới tên người khác.

Góp ý / sửa lỗi: inbox fanpage CodeGym hoặc email info@codegym.vn.

Cảm ơn bạn đã đọc đến đây. Chúc bạn pass phỏng vấn đầu tiên 🍀